Embedded Data Structure
(Lecture Note 8)

Linked List (2)

2025. 3. 27

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of Last Class

KU KONKUK UNIVERSITY

▪ Simple linked list

- **Remember the structure "node"**

  - Two main components : 1) int key; 2) node *next;
  - Do your best to understand meaning of each function (e.g., insert, delete, etc.)
  - Practice yourself to complete the implementation of "main"
  - Memory allocation for storing a specific value

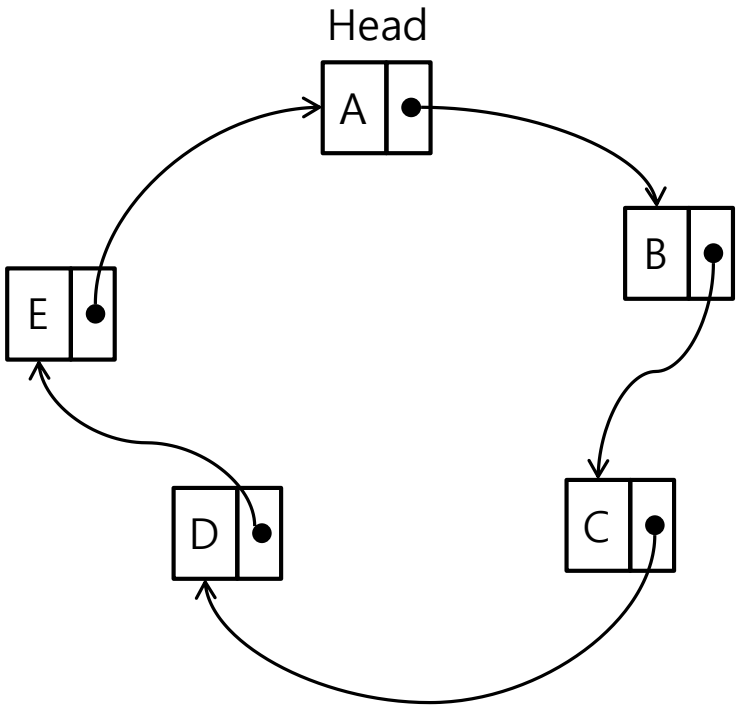
➡ **The most important thing is to draw the pointer on your paper !**

[Example of drawing a simple linked list]

Circular Linked List (1/4)

- **Almost same as the simple linked list except …**
  - **The last node indicates the head node (i.e., no tail)**

    : Just like a circle-shaped linked list !



[Example of drawing a circular linked list]

Circular Linked List (2/4)

- **Joshep's problem using circular linked list**
  - **Example : 10 people sit in a circle (from A to J)**

→ Select a person with 4 interval : E→I→C→H→D→A→J→B→G→F

```
void insert_nodes(int k){
    // make a list from 1 to k
    node *t;
    int i;
    t = (node *)calloc(1, sizeof(node));
    t->key = 1;
    head = t;
    for(i=2; i<=k; i++){
        t->next = [                    ];
        t = t->next;
        t->key = i;
    }
    t->next = head;
}
```

```
void delete_after(node *t){
    node *s;
    s= t->next;
    t->next = t->next->next;
    free(s);

    // just the same as a simple LL
}
```

Circular Linked List (3/4)

- **Joshep's problem using circular linked list – cont'd**
  - **Select a number until n with an interval m**

```
void josephus(int n, int m){
    node *t;
    int i;
    insert_nodes(n);
    t = head;
    printf("₩nAnswer : ");
    while( t != t->next){
        for(i = 0; i<m-1; i++)
            t = t->next;
        printf("%d  ", t->next->key);
        delete_after(t);
    }
    printf("%d", t->key);  // last node
}
```

n : last value, m : interval

Make a circular linked list !

Until nodes remain in a list
(i.e., last node = head)

Circular Linked List (4/4)

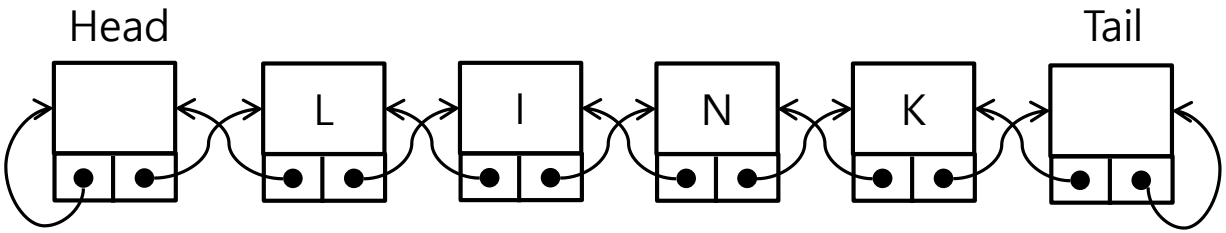
- **Joshep's problem using circular linked list – cont'd**
  - **Complete your main function and check the result !**

```
#include <stdio.h>
[                    ]

void main(){
   int n, m;

   printf("Enter N and M :");
   scanf("%d %d", &n, &m);

   [                    ] ; // Josephus problem
}
```

➡ **Modify this implementation to work for characters (e.g., N=J, M=4) : E→I→C→H→D→A→J→B→G→F**

Doubly Linked List (1/10)

- **Advanced linked list**
  - **There are two pointers for indicating previous and next nodes**



[Example of doubly linked list]

```
typedef struct _dnode
{
    int key;
    struct _dnode *prev;
    struct _dnode *next;
}dnode;
```

What is difference c.p. simple linked ?

- `int key;` → Store data
- `struct _dnode *prev;` → Link for the previous node
- `struct _dnode *next;` → Link for the next node

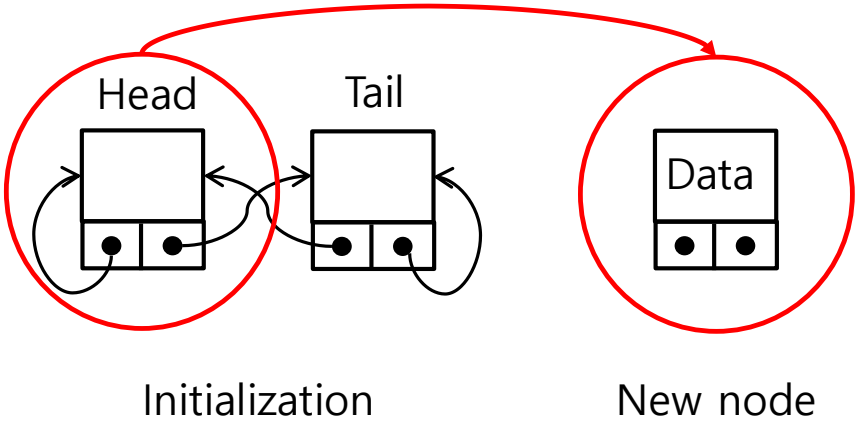
Doubly Linked List (2/10)

▪ C implementation

- **Initialization of doubly linked list**

```
dnode *head, *tail;

void init_dlist()
{
    head = (dnode *)calloc(1, sizeof(dnode));
    tail = (dnode *)calloc(1, sizeof(dnode));
    head->next = tail;
    head->prev = head;
    tail->next = tail;
    tail->prev = head;
}
```

*** Need to control four pointers**



Doubly Linked List (3/10)

- **C implementation – cont'd**
  - **Insert a new node containing value k before node t**

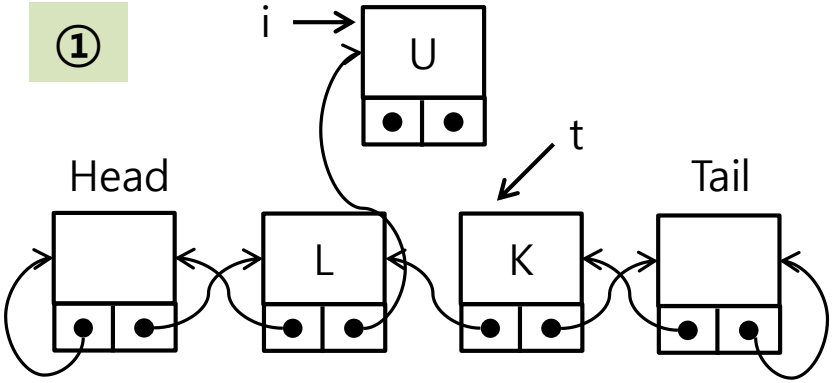
```
dnode *insert_dnode_ptr(int k, dnode *t)
{
    dnode *i;
    if(t == head) return NULL;

    i = (dnode *)calloc(1, sizeof(dnode));
    i->key = t;
    t->prev->next = i;
    i->prev = t->prev;
    t->prev = i;
    i->next = t;

    return i;
}
```

→ The address "i" is assigned to …

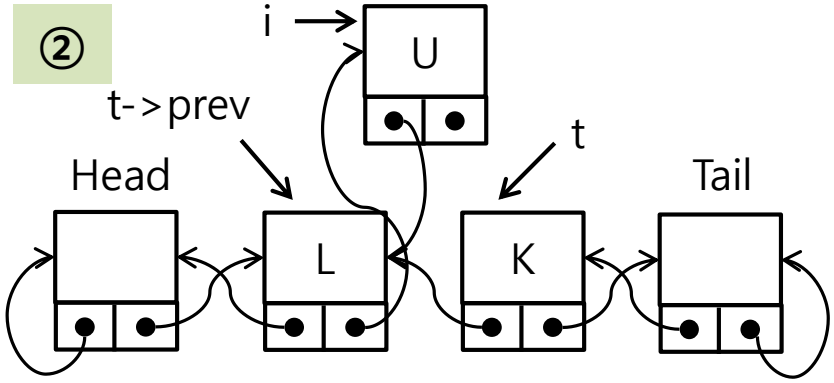
→ Return the inserted node

Doubly Linked List (4/10)

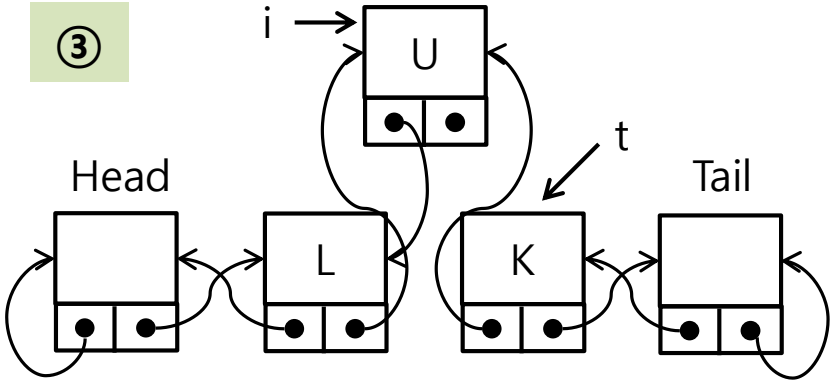
- **C implementation – cont'd**



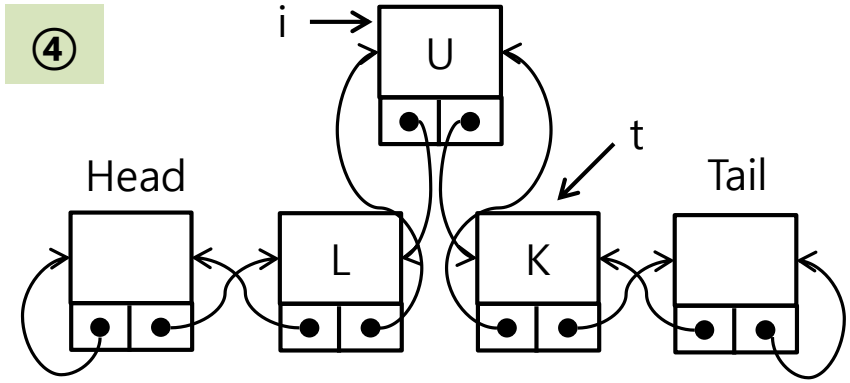
t->prev->next = i



i->prev = t->prev



t->prev = i



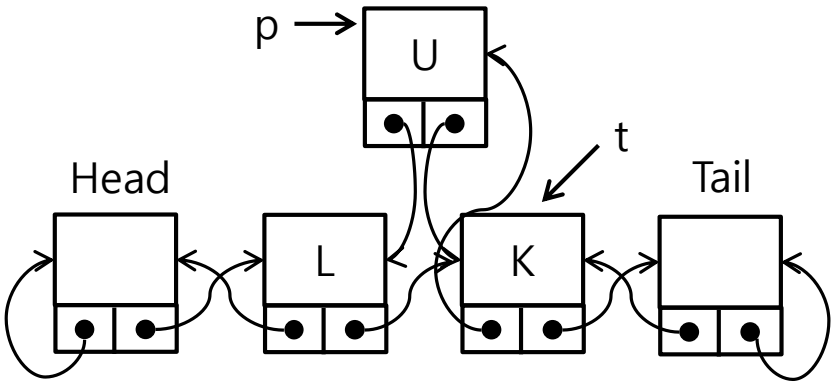
i->next = t

Doubly Linked List (5/10)

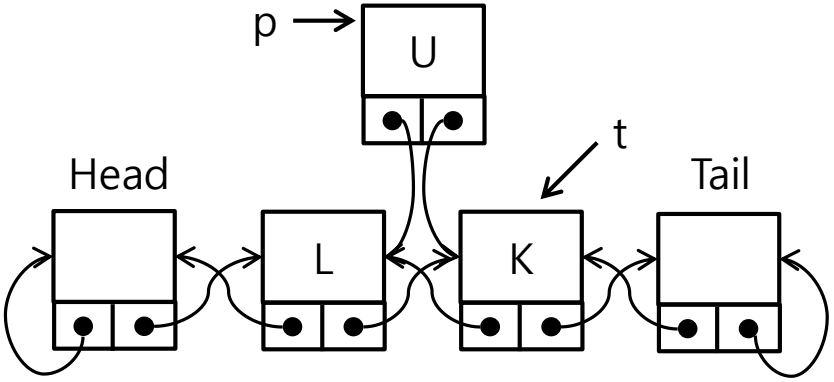
- **C implementation – cont'd**
  - **Delete the given node (simple)**

```
int delete_dnode_ptr(dnode *p)
{
    if(p == head || p == tail)
        return 0;

    p->prev->next = p->next;
    p->next->prev = p->prev;
    free(p);

    return 1;
}
```

Let's think about this flow !



p->prev->next = p->next



p->next->prev= p->prev

Doubly Linked List (6/10)

- **C implementation – cont'd**
  - **Find/delete the given node with a variable "k"**

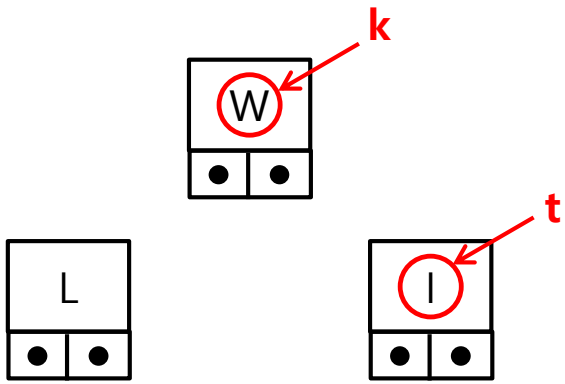
```
dnode *find_dnode(int k)
{
    dnode *s;
    s = head->next;
    while(s->key != k && s != tail)
        s = s->next;
    return s;
}
```

```
int delete_dnode(int k)
{
    dnode *s;
    s = find_dnode(k);

    if(s != tail)
    {
        s->prev->next = s->next;
        s->next->prev = s->prev;
        free(s);
        return 1;
    }
    return 0;
}
```

Doubly Linked List (7/10)

- **C implementation – cont'd**
  - **Insert a node (t앞에 k를 삽입)**

```
dnode *insert_dnode(int k, int t)
{
    dnode *s;
    dnode *i = NULL;
    s = find_dnode(t);
    if(s != tail)
    {
        i = (dnode *)calloc(1, sizeof(dnode));
        i->key = k;
        s->prev->next = i;
        i->prev = s->prev;
        s->prev = i;
        i->next = s;
    }
    return i;
}
```

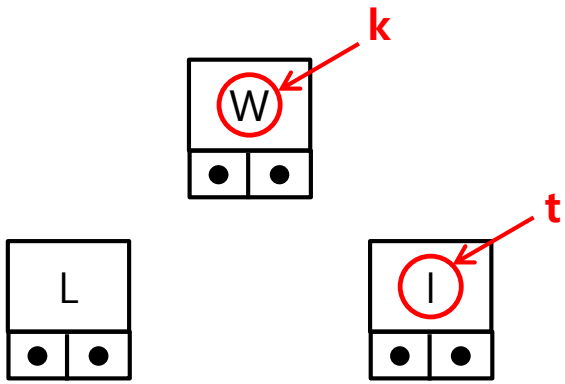


Draw the line step by step !

Doubly Linked List (8/10)

- **C implementation – cont'd**
  - **Ordered sorting in the linked list (dynamic linking)**

```
dnode *ordered_insert(int k)
{
    dnode *s;
    dnode *i;
    s = head->next;
    while(s->key <= k && s != tail)
        s = s->next;
    i = (dnode *)calloc(1, sizeof(dnode));
    i->key = k;
    s->prev->next = i;
    i->prev = s->prev;
    s->prev = i;
    i->next = s;
    return i;
}
```



Draw the line step by step !

Doubly Linked List (9/10)

- **C implementation – cont'd**
  - **Delete all the list and print out the list**

```
void delete_all()
{
    dnode *p;
    dnode *s;
    p = head->next;
    while(p != tail)
    {
        s = p;
        p = p->next;
        free(s);
    }
    head->next = tail;
    tail->prev = head;
}
```

```
void print_dlist(dnode *p)
{
    printf("₩n");
    while(p != tail);
    {
        printf("%-8d", p->key);
        p = p->next;
    }
}
```

Doubly Linked List (10/10)

- **Implementation : check your final results**
  - **Make complete your code as follows:**

```
void main()
{
    dnode *t;

    init_dlist();
    ordered_list(9);
    ordered_list(3);
    ordered_list(5);
    ordered_list(1);
    ordered_list(7);

    printf_list(head->next);

    t = find_dnode(5);
    printf("₩nFinding 5 is %ssuccessful", t == tail ? "un" : "");
    ...
}
```

Make complete your code

1. Insert 4 before 5
2. Inserting node 2 before 9
3. Delete node 4

Summary

- **Advanced linked list**
  - **Circular linked list (without tail) : josephus problem**
  - **Doubly linked list : what is the main difference with SLL ?**

- **Draw your list for better understanding !**